

[Home](#) > [Guide](#) > [Migrating 5](#)[Edit on GitHub](#)

On this page



Upgrade to Express v5

Express 5 is not very different from Express 4; although it maintains the same basic API, there are still changes that break compatibility with the previous version. Therefore, an application built with Express 4 might not work if you update it to use Express 5.

Installation

To install this version, you need to have a **Node.js version 18 or higher**. Then, execute the following command in your application directory:

`npm` `yarn` `pnpm` `bun`

```
npm install "express@5"
```

You can then run your automated tests to see what fails, and fix problems according to the updates listed below. After addressing test failures, run your app to see what errors occur. You'll find out right away if the app uses any methods or properties that are not supported.

Express 5 Codemods

To help you migrate your express server, we have created a set of codemods that will help you automatically update your code to the latest version of Express.

Run the following command for run all the codemods available:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/v5-migration-recipe
```

If you want to run a specific codemod, you can run the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/name-of-the-codemod
```

You can find the list of available codemods [here](#).

Removed methods and properties

If you use any of these methods or properties in your app, it will crash. So, you'll need to change your app after you update to version 5.

app.del()

Express 5 no longer supports the `app.del()` function. If you use this function, an error is thrown. For registering HTTP DELETE routes, use the `app.delete()` function instead.

Initially, `del` was used instead of `delete`, because `delete` is a reserved keyword in JavaScript. However, as of ECMAScript 6, `delete` and other reserved keywords can legally be used as property names.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/route-del-to-delete
```

Or you can update your code manually:

```
-app.del('/user/:id', (req, res) => {  
+app.delete('/user/:id', (req, res) => {  
    res.send(`DELETE /user/${req.params.id}`);  
});
```

app.param(fn)

The `app.param(fn)` signature was used for modifying the behavior of the `app.param(name, fn)` function. It has been deprecated since v4.11.0, and Express 5 no longer supports it at all.

Pluralized method names

The following method names have been pluralized. In Express 4, using the old methods resulted in a deprecation warning. Express 5 no longer supports them at all:

`req.acceptsCharset()` is replaced by `req.acceptsCharsets()`.

`req.acceptsEncoding()` is replaced by `req.acceptsEncodings()`.

`req.acceptsLanguage()` is replaced by `req.acceptsLanguages()`.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/pluralize-method-names
```

Or you can update your code manually:

```
app.all('/', (req, res) => {  
- req.acceptsCharset('utf-8');  
- req.acceptsEncoding('br');  
- req.acceptsLanguage('en');  
+ req.acceptsCharsets('utf-8');  
+ req.acceptsEncodings('br');  
+ req.acceptsLanguages('en');  
  
  // ...  
});
```

Leading colon (:) in the name for `app.param(name, fn)`

A leading colon character (:) in the name for the `app.param(name, fn)` function is a remnant of Express 3, and for the sake of backwards compatibility, Express 4 supported it with a deprecation notice. Express 5 will silently ignore it and use the name parameter without prefixing it with a colon.

This should not affect your code if you follow the Express 4 documentation of [app.param](#), as it makes no mention of the leading colon.

`req.param(name)`

This potentially confusing and dangerous method of retrieving form data has been removed. You will now need to specifically look for the submitted parameter name in the `req.params`, `req.body`, or `req.query` object.

How to update

You can automatically update your code by running the following command:

```
npm    yarn    pnpm    bun
```

```
npx codemod@latest @expressjs/explicit-request-params
```

Or you can update your code manually:

```
app.post('/user', (req, res) => {  
-   const id = req.param('id');  
-   const body = req.param('body');  
-   const query = req.param('query');  
+   const id = req.params.id;  
+   const body = req.body;  
+   const query = req.query;  
  
    // ...  
});
```

res.json(obj, status)

Express 5 no longer supports the signature `res.json(obj, status)`. Instead, set the status and then chain it to the `res.json()` method like this:

```
res.status(status).json(obj).
```

How to update

You can automatically update your code by running the following command:

```
npm    yarn    pnpm    bun
```

```
npx codemod@latest @expressjs/status-send-order
```

Or you can update your code manually:

```
app.post('/user', (req, res) => {  
-   res.json({ name: 'Ruben' }, 201);  
+   res.status(201).json({ name: 'Ruben' });  
});
```

```
});
```

res.jsonp(obj, status)

Express 5 no longer supports the signature `res.jsonp(obj, status)`. Instead, set the status and then chain it to the `res.jsonp()` method like this:

```
res.status(status).jsonp(obj) .
```

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/status-send-order
```

Or you can update your code manually:

```
app.post('/user', (req, res) => {  
-   res.jsonp({ name: 'Ruben' }, 201);  
+   res.status(201).jsonp({ name: 'Ruben' });  
});
```

res.redirect(url, status)

Express 5 no longer supports the signature `res.redirect(url, status)`. Instead, use the following signature: `res.redirect(status, url)`.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/redirect-arg-order
```

Or you can update your code manually:

```
app.get('/user', (req, res) => {  
-   res.redirect('/users', 302);  
+   res.redirect(302, '/users');  
});  
  
// A redirect that relies on the default 302 status is unaffected  
app.get('/admin', (req, res) => {  
    res.redirect('/dashboard');  
});
```

res.redirect('back') and res.location('back')

Express 5 no longer supports the magic string `back` in the `res.redirect()` and `res.location()` methods. Instead, use the `req.get('Referrer') || '/'` value to redirect back to the previous page. In Express 4, the `res.redirect('back')` and `res.location('back')` methods were deprecated.

How to update

You can automatically update your code by running the following command:

```
npm    yarn    pnpm    bun
```

```
npx codemod@latest @expressjs/back-redirect-deprecated
```

Or you can update your code manually:

```
app.get('/user', (req, res) => {  
-   res.redirect('back');  
+   res.redirect(req.get('Referrer') || '/');  
});
```

res.send(body, status)

Express 5 no longer supports the signature `res.send(obj, status)`. Instead, set the status and then chain it to the `res.send()` method like this:

```
res.status(status).send(obj).
```

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/status-send-order
```

Or you can update your code manually:

```
app.get('/user', (req, res) => {  
-   res.send({ name: 'Ruben' }, 200);  
+   res.status(200).send({ name: 'Ruben' });  
});
```

res.send(status)

Express 5 no longer supports the signature `res.send(status)`, where `status` is a number. Instead, use the `res.sendStatus(statusCode)` function, which sets the HTTP response header status code and sends the text version of the code: “Not Found”, “Internal Server Error”, and so on. If you need to send a number by using the `res.send()` function, quote the number to convert it to a string, so that Express does not interpret it as an attempt to use the unsupported old signature.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/status-send-order
```

Or you can update your code manually:

```
app.get('/user', (req, res) => {  
-   res.send(200);  
+   res.sendStatus(200);  
});
```

res.sendFile()

The `res.sendFile()` function has been replaced by a camel-cased version `res.sendFile()` in Express 5.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/camelcase-sendfile
```

Or you can update your code manually:

```
app.get('/user', (req, res) => {  
-   res.sendFile('/path/to/file');  
+   res.sendFile('/path/to/file');  
});
```

res.sendFile() options

The `hidden` and `from` options for `res.sendFile()` are no longer supported. Use `dotfiles` and `root` instead.

The `dotfiles` option applies to hidden directories in the path as well as hidden files. For example, a file served from an absolute path like `/var/www/app/.cache/index.html` now requires `dotfiles: 'allow'`, even though `index.html` is not a dotfile. In Express 4 a hidden directory in the path was served by default; Express 5 returns `404` unless you opt in.

This check only applies to the part of the path that `send` evaluates. When you pass a `root`, only the portion relative to `root` is checked, so a hidden directory inside `root` is unaffected.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/sendfile-options
```

Or you can update your code manually:

```
app.get('/files/:name', (req, res) => {  
-   res.sendFile(req.params.name, { hidden: true, from: '/uploads' });  
+   res.sendFile(req.params.name, { dotfiles: 'allow', root: '/uploads' });  
});
```

If you serve an absolute path that contains a hidden directory, opt in with `dotfiles: 'allow'` or use `root` so the hidden segment is not part of the evaluated path:

```
app.get('/build', (req, res) => {  
-   res.sendFile('/var/www/app/.cache/index.html');  
+   res.sendFile('/var/www/app/.cache/index.html', { dotfiles: 'allow' });  
+   // or: res.sendFile('index.html', { root: '/var/www/app/.cache' });  
});
```

express.static() options

The `hidden` and `from` options for `express.static()` are no longer supported. Use `dotfiles` and `root` instead. Note that `from` was never documented in the API but was

accepted as an alias for `root`. The default value of `dotfiles` is now `"ignore"`.

The `dotfiles` check now also applies to hidden **directories** in the request path, not just hidden files. A request like `GET /.well-known/acme-challenge/...` that was served by default in Express 4 now returns `404` unless you set `dotfiles: 'allow'`. This only affects the part of the path relative to the configured `root`; a hidden directory in `root` itself is unaffected.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/static-dotfiles
```

Or you can update your code manually:

```
const express = require('express');
const app = express();

- app.use(express.static('public', { hidden: true }));
+ app.use(express.static('public', { dotfiles: 'allow' }));
```

If you rely on serving a hidden directory such as `.well-known` (for example, ACME/Let's Encrypt challenges), opt in explicitly. This is needed even if you never used the `hidden` option:

```
- app.use(express.static('public'));
+ app.use(express.static('public', { dotfiles: 'allow' }));
```

router.param(fn)

The `router.param(fn)` signature was used for modifying the behavior of the `router.param(name, fn)` function. It has been deprecated since v4.11.0, and Express 5 no longer supports it at all.

express.static.mime

In Express 5, `mime` is no longer an exported property of the `static` field. Use the [mime-types package](#) to work with MIME type values.

How to update

You can automatically update your code by running the following command:

npm yarn pnpm bun

```
npx codemod@latest @expressjs/static-mime
```

Or you can update your code manually:

```
-express.static.mime.lookup('json');  
+const mime = require('mime-types');  
+mime.lookup('json');
```

MIME type changes

Several MIME types have changed due to updates in [mime-db](#). These changes only affect `express.static()` and `res.sendFile()`. For a full list of changes, see the [mime-db changelog](#).

Express 4 uses `mime-db` version **1.52.0**, while Express 5 uses newer versions that reflect updates to IANA and other MIME type specifications. The most notable change is that JavaScript files (`.js`) are now served as `text/javascript` instead of `application/javascript`.

In Express 5, changes to MIME types from `mime-db` updates are not considered breaking changes, so be careful when updating your dependencies as MIME types may change between minor or patch versions.

express:router debug logs

Router handling logic is now performed by a separate dependency ([router](#)) maintained by the Express team, so debug logs have moved to a different namespace. Before Express 5.1, these debug logs did not exist. To get them, update to a recent Express 5 version or update the `router` package in your `package-lock.json`:

v4	v5
<code>express:router</code>	<code>router</code>
<code>express:router:layer</code>	<code>router:layer</code>
<code>express:router:route</code>	<code>router:route</code>
<code>express:*</code> (includes all)	<code>express:* + router + router:*</code>

How to update

```
-DEBUG=express:* node index.js
+DEBUG=express:*,router,router:* node index.js
```

Changed

These APIs still exist but their behavior has changed. Review these changes to make sure your app works as expected.

Path route matching syntax

Path route matching syntax is when a string is supplied as the first parameter to the `app.all()`, `app.use()`, `app.METHOD()`, `router.all()`, `router.METHOD()`, and `router.use()` APIs. The following changes have been made to how the path string is matched to an incoming request:

- The wildcard `*` must have a name, matching the behavior of parameters `:`, use `/*splat` instead of `/*`

```
-app.get('/*', async (req, res) => {
+app.get('/*splat', async (req, res) => {
```

```
res.send('ok');
});
```

Note

`*splat` matches any path without the root path. If you need to match the root path as well `/`, you can use `{*splat}`, wrapping the wildcard in braces.

```
app.get('/{*splat}', async (req, res) => {
  res.send('ok');
});
```

- The optional character `?` is no longer supported, use braces instead.

```
-app.get('/:file.:ext?', async (req, res) => {
+app.get('/:file{.:ext}', async (req, res) => {
  res.send('ok');
});
```

- Regexp characters are not supported. For example:

```
-app.get('/[discussion|page]/:slug', async (req, res) => {
+app.get(['/discussion/:slug', '/page/:slug'], async (req, res) => {
  res.status(200).send('ok');
});
```

- Some characters have been reserved to avoid confusion during upgrade `( ) [ ] ? + !`, use `\` to escape them.
- Parameter names now support valid JavaScript identifiers, or quoted like `:"this"`.

Rejected promises handled from middleware and handlers

Request middleware and handlers that return rejected promises are now handled by forwarding the rejected value as an `Error` to the error handling middleware. This means that using `async` functions as middleware and handlers are easier than ever. When an error is

thrown in an `async` function or a rejected promise is `await` ed inside an `async` function, those errors will be passed to the error handler as if calling `next(err)` .

Details of how Express handles errors is covered in the [error handling documentation](#).

How to update

You can now use `async/await` directly without manually catching errors. If `getUserById` throws an error or rejects, `next` will be called automatically with the rejected value.

```
- app.get('/user/:id', (req, res, next) => {  
-   getUserById(req.params.id)  
-     .then((user) => res.send(user))  
-     .catch(next);  
- });  
+ app.get('/user/:id', async (req, res) => {  
+   const user = await getUserById(req.params.id);  
+   res.send(user);  
+ });
```

express.urlencoded

The `express.urlencoded` method makes the `extended` option `false` by default.

How to update

If your application relies on the `extended` behavior, explicitly set it to `true` :

```
- app.use(express.urlencoded());  
+ app.use(express.urlencoded({ extended: true }));
```

express.static dotfiles

The `express.static` middleware's `dotfiles` option now defaults to `"ignore"` . In Express 4, dotfiles were served by default. As a result, files inside a directory that starts with a dot (`.`), such as `.well-known` , will no longer be accessible and will return a **404 Not Found** error. This can break functionality that depends on serving dot-directories, such as Android App Links and Apple Universal Links.

How to update

Serve specific dot-directories explicitly using the `dotfiles: "allow"` option. This allows you to safely serve only the intended dot-directories while keeping the default secure behavior for other dotfiles.

```
+app.use('/.well-known', express.static('public/.well-known', { dotfiles: 'allow'
  app.use(express.static('public')));
```

router.param() with an array of names

`router.param(name, fn)` no longer accepts an array for `name`. In Express 4 an array was accepted silently; in Express 5, passing anything other than a string throws `TypeError: argument name must be a string`. (Note that `app.param()` still accepts an array of names.)

How to update

Register each parameter name with its own `router.param()` call:

```
-router.param(['id', 'page'], (req, res, next, value) => {
-  // ...
-});
+const loadParam = (req, res, next, value) => {
+  // ...
+};
+router.param('id', loadParam);
+router.param('page', loadParam);
```

app.listen

In Express 5, the `app.listen` method will invoke the user-provided callback function (if provided) when the server receives an error event. In Express 4, such errors would be thrown. This change shifts error-handling responsibility to the callback function in Express 5. If there is an error, it will be passed to the callback as an argument. For example:

```
const server = app.listen(8080, '0.0.0.0', (error) => {
+  if (error) {
```

```
+   throw error; // e.g. EADDRINUSE
+ }
  console.log(`Listening on ${JSON.stringify(server.address())}`);
});
```

app.router

The `app.router` object, which was removed in Express 4, has made a comeback in Express 5. In the new version, this object is just a reference to the base Express router, unlike in Express 3, where an app had to explicitly load it.

req.body

The `req.body` property returns `undefined` when the body has not been parsed. In Express 4, it returns `{}` by default.

```
app.post('/user', (req, res) => {
  console.dir(req.body);
  // Express 4
- // => {}
  // Express 5
+ // => undefined
});
```

req.host

In Express 4, the `req.host` function incorrectly stripped off the port number if it was present. In Express 5, the port number is maintained.

req.params

The `req.params` object now has a **null prototype** when using string paths. However, if the path is defined with a regular expression, `req.params` remains a standard object with a normal prototype. Additionally, there are two important behavioral changes:

Wildcard parameters are now arrays:

Wildcards (e.g., `/*splat`) capture path segments as an array instead of a single string.

```
app.get('/*splat', (req, res) => {
  // GET /foo/bar
  console.dir(req.params);
  // => [Object: null prototype] { splat: [ 'foo', 'bar' ] }
});
```

Unmatched parameters are omitted:

In Express 4, unmatched wildcards were empty strings (`' '`) and optional `:` parameters (using `?`) had a key with value `undefined` . In Express 5, unmatched parameters are completely omitted from `req.params` .

```
-// v4: unmatched wildcard is empty string
-app.get('/*', (req, res) => {
-  // GET /
-  console.dir(req.params);
-  // => { '0': '' }
-});
-
-// v4: unmatched optional param is undefined
-app.get('/:file.:ext?', (req, res) => {
-  // GET /image
-  console.dir(req.params);
-  // => { file: 'image', ext: undefined }
-});
-
+// v5: unmatched optional param is omitted
+app.get('/:file{.:ext}', (req, res) => {
+  // GET /image
+  console.dir(req.params);
+  // => [Object: null prototype] { file: 'image' }
+});
```

req.query

The `req.query` property is no longer a writable property and is instead a getter. The default query parser has been changed from “extended” to “simple”.

```
app.get('/search', (req, res) => {
```

```
// This is no longer possible in Express 5
- req.query.page = 1;
});
```

res.clearCookie

The `res.clearCookie` method ignores the `maxAge` and `expires` options provided by the user.

```
app.get('/logout', (req, res) => {
- res.clearCookie('session', { maxAge: 0, expires: new Date(0) });
+ res.clearCookie('session');
});
```

res.status

The `res.status` method only accepts integers in the range of 100 to 999, following the behavior defined by Node.js, and it returns an error when the status code is not an integer.

```
app.get('/user', (req, res) => {
- res.status(99); // Throws an error
  res.status(200); // OK
});
```

res.vary

The `res.vary` throws an error when the `field` argument is missing. In Express 4, if the argument was omitted, it gave a warning in the console.

```
app.get('/user', (req, res) => {
- res.vary(); // Throws an error
  res.vary('Accept'); // OK
});
```

Improvements

These changes don't require any migration steps, but are worth knowing about when upgrading.

`res.render()`

This method now enforces asynchronous behavior for all view engines, avoiding bugs caused by view engines that had a synchronous implementation and that violated the recommended interface.

Brotli encoding support

Middleware like `express.json()`, `express.urlencoded()`, `express.text()`, and `express.raw()` now support Brotli (Content-Encoding: br) decompression for incoming request bodies, in addition to `gzip` and `deflate`.

Previous



Moving to Express 4

Next



Building template engines

Copyright [OpenJS Foundation](#) and Express.js contributors. All rights reserved. The [OpenJS Foundation](#) has registered trademarks and uses trademarks. For a list of trademarks of the [OpenJS Foundation](#), please see our [Trademark Policy](#) and [Trademark List](#). Trademarks and logos not indicated on the [list of OpenJS Foundation trademarks](#) are trademarks™ or registered® trademarks of their respective holders. Use of them does not imply any affiliation with or endorsement by them.

[The OpenJS Foundation](#) [AI Coding Assistants Policy](#) [Bylaws](#) [Code of Conduct](#) [Cookie Policy](#) [Privacy Policy](#) [Security Policy](#) [Terms of Use](#) [Trademark List](#) [Trademark Policy](#)



Kawaii?

